

Week 2 Report

Benchmarking Teacher-Reference LSTM, Proposed LSTM, and tsai ResNet on the Processed Z24 Dataset

TRAN VAN PHUONG

20 September 2026

Abstract

This report evaluates a teacher-reference 1DCNN–LSTM–ResNet, a proposed 1DCNN–LSTM–ResNet, and a tsai ResNet on the same processed Z24 dataset. All experiments use 1,530 samples, 17 balanced classes, the same setup-grouped train/validation/test split, and per-sensor Z-score normalization calculated from the training set only. The teacher-reference pipeline preserves the lecturer’s sensor-first input convention and softmax recurrent activation, whereas the proposed pipeline treats the 6,000 measurements as time steps, uses a sigmoid recurrent activation, and adds Dropout, L2 regularization, AdamW, `tf.data`, and two-GPU training. The tsai pipeline uses a one-dimensional ResNet implemented with PyTorch/fastai and trains on one GPU.

The proposed pipeline increased test accuracy from 56.18% to 60.00% and test macro F1 from 53.63% to 55.02%. Its train–test accuracy gap decreased from 43.82 to 33.14 percentage points, indicating less severe overfitting. However, the improvement was not uniform across classes and amounted to only 13 additional correct predictions among 340 test samples. The tsai ResNet obtained 60.29% test accuracy and 57.67% macro F1. It therefore classified only one more test sample correctly than the proposed LSTM, but improved macro F1 by 2.65 percentage points, indicating a different class-level trade-off. The proposed LSTM also required 8.35 seconds per steady-state epoch, compared with 0.78 seconds for the teacher-reference pipeline and 8.05 seconds for tsai. The main reason for the LSTM cost is that it processes approximately 3,000 recurrent steps instead of approximately 14. Finally, the recorded benchmark policy was `float32`; therefore, this run does not support a claim that mixed precision accelerated the proposed model.

1 Objective and scope

The objective is to determine how the complete pipeline and model changes affected:

1. classification performance on train, validation, and test data;
2. overfitting and generalization to unseen setups;
3. model size and training speed, including the tsai ResNet baseline;
4. class-specific errors; and
5. the validity of claims concerning GPU acceleration.

The evaluated notebooks are:

Proposed LSTM and tsai ResNet:
`shm-ml-new.ipynb`

Teacher-reference pipeline:
`shm-old-lstm.ipynb`

The first notebook contains two independent experiments: the proposed LSTM in TensorFlow/Keras and the tsai ResNet in PyTorch/fastai. The teacher-reference notebook is an adaptation of the

lecturer’s source code to the processed dataset. It is not the unmodified original experiment. The original code used pickle files, an input shape of (5, 8000), and a validation set containing training samples. For the benchmark reported here, the teacher architecture and training style were retained, but all three pipelines used the same processed data, grouped split, and training-only normalization. This distinction is essential: the results below compare three trained models on controlled data, not the new models against the accuracy originally reported by the lecturer.

2 Controlled dataset and evaluation protocol

The processed dataset has the following contract:

```
inputs: (1530, 27, 6000), float32
labels: (1530,), int64
classes: 17
layout: samples x sensors x time samples
```

All three experiments used the same seed-42 setup allocation:

Table 1: Identical grouped split used by all three pipelines.

Split	Segments	Recordings	Setup IDs
Train	1,020	102	3, 0, 7, 2, 4, 6
Validation	170	17	1
Test	340	34	5, 8

All ten segments from one condition/setup recording remain in one split. Thus, the sets of recording identifiers satisfy:

$$R_{\text{train}} \cap R_{\text{validation}} =, \quad R_{\text{train}} \cap R_{\text{test}} =, \quad R_{\text{validation}} \cap R_{\text{test}} = .$$

Every class contributes 60 train, 10 validation, and 20 test segments. Because the test set is class-balanced, macro recall is numerically equal to accuracy in this experiment. It should not be interpreted as independent confirmation of the accuracy result.

All three pipelines apply per-sensor Z-score normalization:

$$x'_{i,s,t} = \frac{x_{i,s,t} - \mu_{s,\text{train}}}{\sigma_{s,\text{train}}}.$$

Only training samples are used to calculate $\mu_{s,\text{train}}$ and $\sigma_{s,\text{train}}$. The resulting statistics are then applied to train, validation, and test data, avoiding preprocessing leakage.

3 Pipeline changes

The core Conv1D–ResNet–LSTM structure is preserved. However, the proposed pipeline is not a one-variable modification. It changes the input semantics, LSTM gate activation, optimizer, regularization, batch size, model-selection criterion, data loader, and distribution strategy simultaneously. The measured difference is therefore the effect of the complete pipeline package; it cannot be attributed to any one component without an ablation study.

Table 2: Configuration of the two benchmarked pipelines.

Component	Teacher-reference	Proposed
Stored data	(1530, 27, 6000)	(1530, 27, 6000)
Model input	(27, 6000)	(6000, 27)
Input interpretation	27 sensor positions as sequence steps; 6,000 features per step	6,000 time samples as sequence steps; 27 sensor features per step
Sequence after stride-2 Conv1D	Approximately 14 steps	3,000 steps
LSTM	128 units, recurrent softmax	128 units, recurrent sigmoid
Parameters	3,949,137	508,689
Normalization	Per-sensor train-only Z-score	Per-sensor train-only Z-score
Regularization	No explicit Dropout/L2	Dropout, L2, and decoupled weight decay
Optimizer	Adam	AdamW
Batch size	64	32
Model selection	Maximum validation accuracy, patience 30	Minimum validation loss, patience 15
Data input	Direct NumPy arrays	<code>tf.data</code> with shuffle and prefetch
GPU execution	One replica on GPU 0	Two replicas using MirroredStrategy
Recorded precision policy	<code>float32</code>	<code>float32</code>

3.1 Proposed LSTM versus tsai ResNet

The proposed LSTM and tsai experiments share the same dataset, setup IDs, per-sensor normalization, batch size, and reported metrics. Their input tensors use different framework conventions but represent the same physical axes:

Keras LSTM: (batch, time, sensors) = (N, 6000, 27)
 tsai ResNet: (batch, channels, time) = (N, 27, 6000)

Thus, unlike the teacher-reference layout, both models treat 6,000 as the time dimension. Their training configurations are summarized below.

The tsai ResNet contains no recurrent layer. Its temporal convolutions can be computed in parallel across the signal, whereas the LSTM state at step t depends on the state at step $t - 1$. This architectural distinction is central to the speed comparison.

3.2 Change in temporal representation

The teacher-reference flow is:

```
(batch, 27, 6000)
  |
  v
stride-2 Conv1D
  |
  v
(batch, 14, 64)
  |
  v
ResNet and LSTM over approximately 14 steps
```

Table 3: Configuration of the proposed LSTM and tsai ResNet.

Component	Proposed LSTM	tsai ResNet
Framework	TensorFlow/Keras	PyTorch, fastai, and tsai
Model	1DCNN-LSTM-ResNet	One-dimensional ResNet
Input convention	(N, 6000, 27)	(N, 27, 6000)
Parameters	508,689	493,713
Batch size	32	32
Optimizer	AdamW	Adam with one-cycle schedule
Regularization	Dropout, L2, weight decay	Weight decay 10^{-4}
Epoch control	Maximum 100; early stopping	Fixed 20 epochs
Model restoration	Minimum validation loss	No best-epoch restoration
Recorded precision	Float32	Float32
Training devices	Two T4 replicas	One T4, cuda:0

The proposed flow is:

```
(batch, 6000, 27)
  |
  v
stride-2 Conv1D
  |
  v
(batch, 3000, 64)
  |
  v
ResNet and LSTM over 3000 steps
```

The proposed layout is more consistent with the physical meaning of the data: the recurrent dimension follows time, while the 27 simultaneous sensor values form the feature vector at each time step. The cost is a much longer recurrent sequence.

3.3 Change in capacity and regularization

The proposed model reduced the parameter count by:

$$1 - \frac{508,689}{3,949,137} = 87.12\%.$$

Despite containing fewer parameters, it performs more sequential work because the LSTM sequence grows from approximately 14 to 3,000 steps. Dropout, L2, and AdamW further distinguish it from the unregularized teacher-reference model. These mechanisms are expected to lower training accuracy while potentially improving validation and test performance.

4 Experimental results

4.1 Classification metrics

The proposed pipeline correctly classified 204 of 340 test samples, compared with 191 for the teacher-reference pipeline. This is a gain of 13 correct predictions. The accuracy and precision improvements are meaningful enough to motivate further experiments, but the smaller macro-F1 gain shows that the benefit is not evenly distributed among classes.

Table 4: Measured final metrics for the three completed runs.

Split	Metric	Teacher-ref.	Proposed LSTM	tsai ResNet
Train	Accuracy	100.00%	93.14%	100.00%
Train	Precision macro	100.00%	94.23%	100.00%
Train	Recall macro	100.00%	93.14%	100.00%
Train	F1 macro	100.00%	92.76%	100.00%
Validation	Accuracy	54.71%	58.82%	63.53%
Validation	Precision macro	50.72%	50.29%	60.14%
Validation	Recall macro	54.71%	58.82%	63.53%
Validation	F1 macro	48.93%	51.49%	56.77%
Test	Accuracy	56.18%	60.00%	60.29%
Test	Precision macro	56.67%	60.93%	58.81%
Test	Recall macro	56.18%	60.00%	60.29%
Test	F1 macro	53.63%	55.02%	57.67%

Compared directly with the proposed LSTM, tsai increased validation accuracy by 4.71 percentage points and validation macro F1 by 5.28 percentage points. On test data, it increased accuracy by only 0.29 percentage points, equivalent to one additional correct sample, while test macro F1 increased by 2.65 percentage points. Its test macro precision was 2.12 percentage points lower. Consequently, the two new models have effectively tied test accuracy, while tsai provides the better macro F1 in this run.

4.2 Overfitting

For accuracy, the train–test gaps are:

$$G_{\text{teacher}} = 100.00 - 56.18 = 43.82 \text{ percentage points,}$$

$$G_{\text{proposed}} = 93.14 - 60.00 = 33.14 \text{ percentage points.}$$

For tsai:

$$G_{\text{tsai}} = 100.00 - 60.29 = 39.71 \text{ percentage points.}$$

The proposed pipeline reduces the gap by 10.68 percentage points. The teacher-reference model reached approximately 100% training accuracy by epoch 10 while validation accuracy remained near 50–55%, which is strong evidence of memorization. The proposed model still overfits: training loss continues to decrease after validation loss reaches its minimum, and validation accuracy fluctuates considerably. Nevertheless, its lower training score and higher test score indicate that the regularized pipeline generalizes better in this run.

The tsai model also reached 100% train accuracy. Its 39.71-point train–test gap is smaller than the teacher-reference gap but 6.57 points larger than the proposed LSTM gap. Thus, tsai obtains the highest validation metrics and macro F1, but the proposed LSTM shows the smallest accuracy-based overfitting gap.

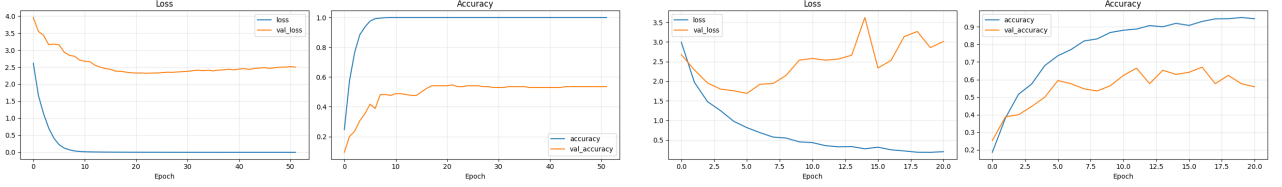


Figure 1: Training histories exported from the Kaggle notebooks. The teacher-reference model (left) reaches 100% training accuracy rapidly while validation accuracy plateaus near 55%. The proposed LSTM (right) learns more gradually, but validation loss begins increasing after its early minimum.

4.3 Model-selection difference

The proposed run stopped after 21 epochs and restored the epoch selected by minimum validation loss. Its minimum validation loss occurred near epoch 6. The training log later reached a maximum validation accuracy of 67.06% at epoch 17, but those weights were not selected because validation loss had worsened.

The teacher-reference run monitored validation accuracy, reached its best value of 54.71% near epoch 22, and stopped after 52 epochs. Thus, final model selection is not controlled between the two LSTM experiments. The proposed model’s final validation accuracy of 58.82% must not be confused with the transient 67.06% observed during training. A fair architecture comparison should use the same monitored metric and patience in both LSTM pipelines. For a class-balanced classification problem, validation macro F1 is also a reasonable selection criterion.

The tsai run used a fixed 20-epoch one-cycle schedule without early stopping or best-weight restoration. Its validation accuracy reached 72.35% at epoch 12 but ended at 63.53% after epoch 20. Therefore, all three pipelines use different model-selection rules. The reported tsai result corresponds to the last epoch, not the epoch with maximum validation accuracy.

5 Class-specific effects

Each class contains 20 test samples. Table 5 summarizes the changes visible in the two test confusion matrices.

Table 5: Largest class-level changes in correct test predictions.

Class	Teacher-ref. correct	Proposed correct	Change
16	0/20	20/20	+20
4	9/20	20/20	+11
14	3/20	6/20	+3
8	0/20	2/20	+2
2	11/20	12/20	+1
5	20/20	17/20	−3
3	7/20	2/20	−5
0	18/20	12/20	−6
12	10/20	0/20	−10

The net gain is strongly influenced by classes 16 and 4. At the same time, class 12 falls from 50% recall to 0%, and class 15 remains at 0/20 for both models. Consequently, the proposed pipeline should not be described as a uniform improvement across all damage conditions. The persistent failure on class 15 and the trade-off between classes 12 and 16 require investigation of signal similarity, label definitions, and per-class feature distributions.

The tsai and proposed-LSTM confusion matrices are strongly complementary. Table 6 reports the largest differences.

Table 6: Class-level comparison of tsai and the proposed LSTM.

Class	Proposed LSTM correct	tsai correct	tsai – LSTM
12	0/20	12/20	+12
2	12/20	20/20	+8
14	6/20	10/20	+4
5	17/20	20/20	+3
3	2/20	0/20	–2
16	20/20	0/20	–20

The proposed LSTM recognizes all class-16 samples but none from class 12; tsai recognizes 12 class-12 samples but none from class 16. Both models fail on all class-15 samples. This pattern explains why their overall accuracies are nearly identical despite very different predictions. It also motivates a future confidence-level ensemble experiment, although an ensemble improvement cannot be assumed without paired prediction analysis.

6 Training-speed results

Table 7: Measured end-to-end training performance on Kaggle T4 GPUs.

Measurement	Teacher-reference	Proposed LSTM	tsai ResNet
Visible GPUs	2 Tesla T4	2 Tesla T4	2 Tesla T4
Training replicas	1	2	1
Completed epochs	52	21	20
Total training time	66.29 s	190.81 s	160.95 s
Mean epoch time	1.24 s	8.98 s	8.04 s
Mean epoch time excluding first	0.78 s	8.35 s	8.05 s
Effective throughput (samples/s)	1,308.42	122.19	126.69

The proposed pipeline is approximately:

$$\frac{8.3476}{0.7796} \approx 10.7$$

times slower per steady-state epoch. Its complete run is approximately 2.88 times longer, despite stopping after fewer than half as many epochs.

This result does not contradict cuDNN optimization. The teacher-reference LSTM processes only about 14 recurrent steps, whereas the proposed LSTM processes 3,000. The sequence-length ratio is approximately:

$$\frac{3000}{14} \approx 214.$$

The proposed model uses the more natural temporal orientation, but this choice substantially increases computation. Moreover, its global batch of 32 is split across two GPUs, leaving approximately 16 samples per replica. Gradient all-reduce and model-replication overhead can reduce the benefit of

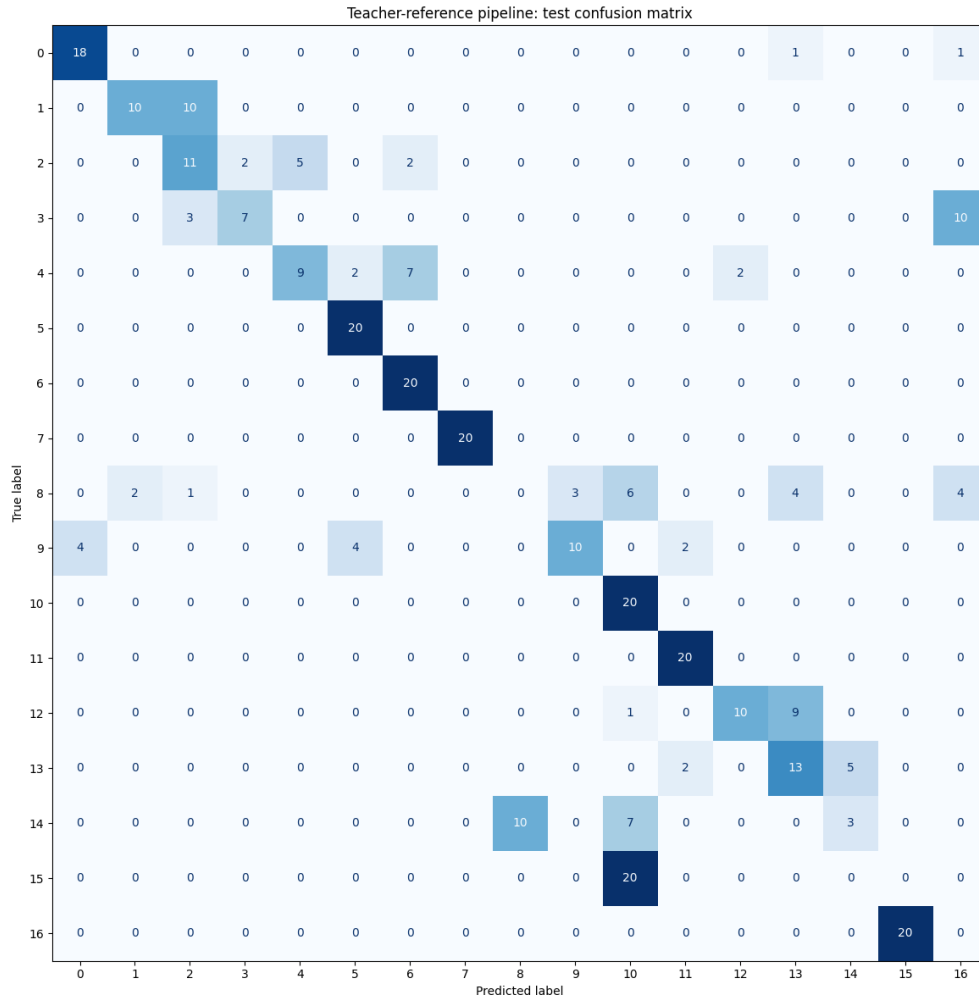


Figure 2: Test confusion matrix of the teacher-reference LSTM. The model is perfect on several classes but completely misses classes 8, 15, and 16.

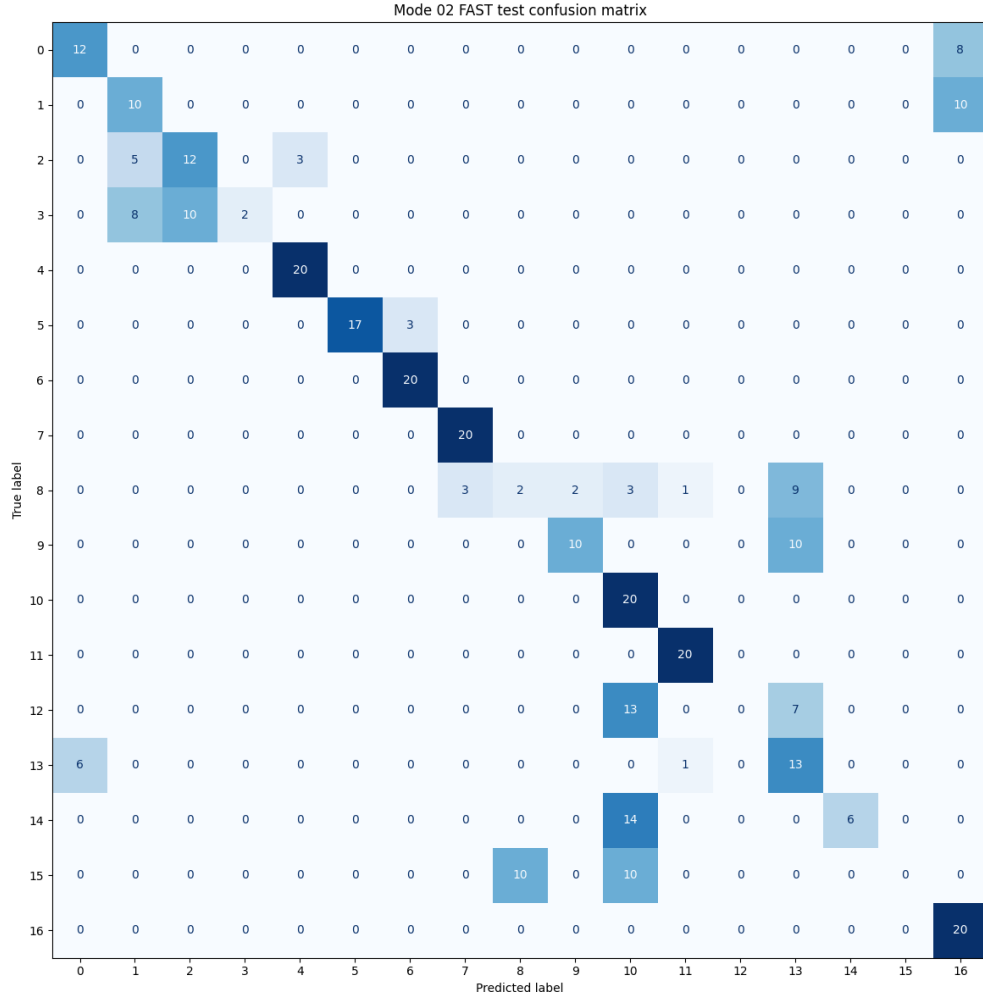


Figure 3: Test confusion matrix of the proposed LSTM. Classes 4 and 16 improve substantially, while class 12 falls to zero correct predictions.

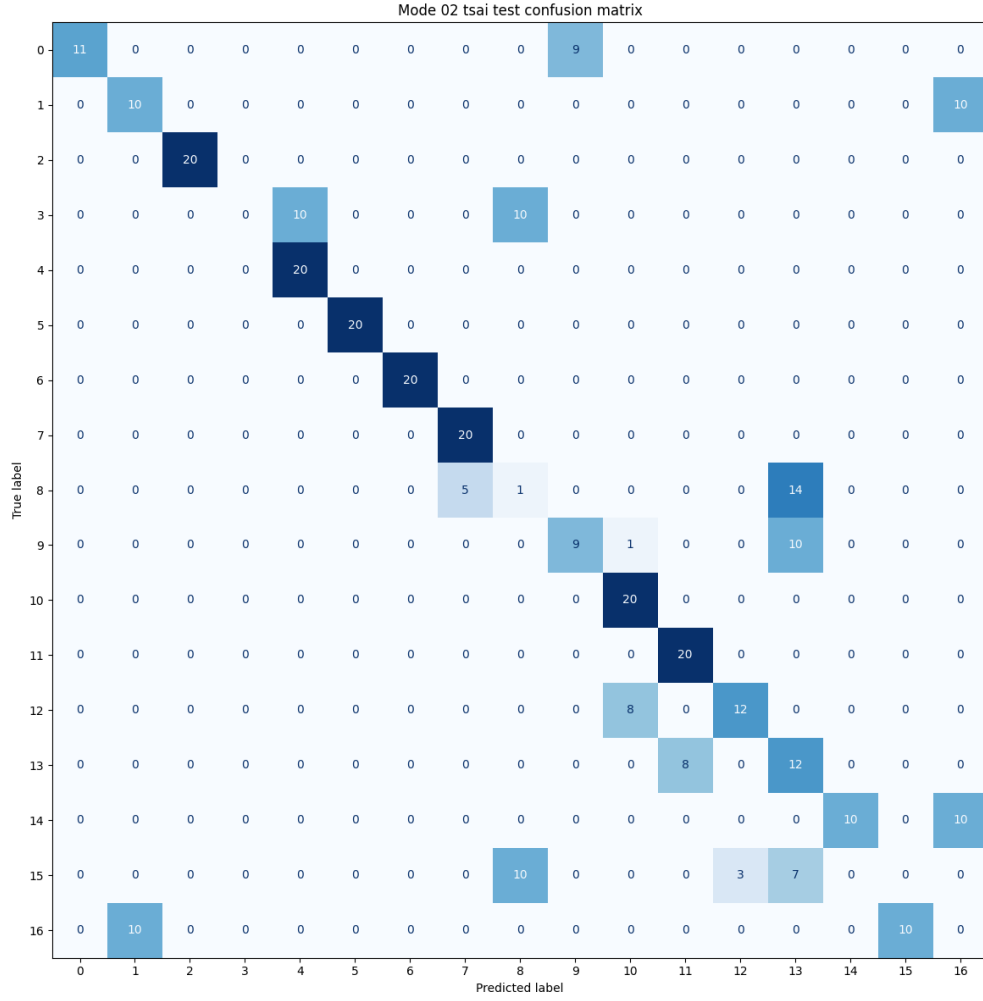


Figure 4: Test confusion matrix of the tsai ResNet. It recovers class 12 and performs strongly on classes 2, 4–7, 10, and 11, but misses classes 3, 15, and 16.

the second GPU at this dataset size. Therefore, the experimental result is that the new pipeline is slower, not faster, than the teacher-reference adaptation.

The tsai ResNet is approximately 3.55% faster per steady-state epoch than the proposed LSTM and processes approximately 3.68% more samples per second. Its total run is 29.86 seconds shorter, although this comparison includes 20 fixed tsai epochs versus 21 early-stopped LSTM epochs. More importantly, tsai obtains this throughput on one T4 while the LSTM uses two replicas. Convolutional operations can exploit parallelism across the complete time axis, whereas the LSTM retains recurrent dependencies across 3,000 steps. Nevertheless, a pure architecture-speed comparison should rerun both models on one GPU with the same precision and batch size.

7 Mixed-precision audit

The beginning of the proposed notebook printed:

```
Mixed precision policy: mixed_float16
```

However, its saved benchmark summary reported:

```
precision_policy: float32
```

The notebook calls `clear_session()` after setting the mixed-precision policy and before constructing the model. In the installed Keras version, `clear_session()` resets global Keras state, including the global dtype policy. The recorded result is therefore consistent with the model being built under `float32`. This experiment must not be used as evidence that Tensor Cores or mixed precision produced the measured result.

The intended order should be:

```
tf.keras.backend.clear_session()
tf.keras.mixed_precision.set_global_policy(
    'mixed_float16' if gpus else 'float32'
)

with strategy.scope():
    model = build_model()

print('Policy_after_model_creation:',
      tf.keras.mixed_precision.global_policy().name)
print('First_Conv1D_policy:', model.layers[1].dtype_policy)
```

The output classification layer should remain explicitly `float32` for numerical stability. After correcting the order, the proposed pipeline must be rerun before reporting a mixed-precision speedup.

8 What can and cannot be concluded

8.1 Supported conclusions

For the exact three runs examined here:

- The proposed complete pipeline improves test accuracy by 3.82 percentage points and macro F1 by 1.39 percentage points.
- It reduces the train-test accuracy gap by 10.68 percentage points.
- It uses 87.12% fewer parameters.
- It is approximately 10.7 times slower per steady-state epoch.

- Its improvement is concentrated in a subset of classes rather than being uniform across all 17 classes.
- The tsai ResNet and proposed LSTM have nearly identical test accuracy: 60.29% and 60.00%, respectively.
- Tsai improves test macro F1 by 2.65 percentage points over the proposed LSTM, but has a 6.57-point larger train–test accuracy gap.
- Tsai is approximately 3.55% faster per steady-state epoch while using one GPU instead of the LSTM’s two replicas.
- Tsai and the LSTM make complementary class-level errors, especially for classes 12 and 16.

8.2 Unsupported causal claims

The experiment does not establish that the accuracy gain was caused specifically by sigmoid, AdamW, Dropout, the input transpose, or any other single change. All of these components changed together. It also does not show that mixed precision accelerated training, because the recorded policy was `float32`. The tsai–LSTM comparison also does not isolate architecture alone because the frameworks, optimizer schedules, device counts, and model-selection procedures differ. Finally, one training seed and one grouped split are insufficient for a strong claim of general superiority.

9 Recommended follow-up experiments

The following sequence would isolate the contribution of each modification:

1. Use the same grouped split, Z-score normalization, batch size, GPU count, precision, early-stopping metric, and patience for every run.
2. Start from the teacher-reference pipeline and change only the input orientation from (27, 6000) to (6000, 27).
3. Change only the LSTM recurrent activation from softmax to sigmoid.
4. Add Dropout, then L2, then AdamW in separate ablations.
5. Evaluate one-GPU and two-GPU execution separately.
6. Add identical early stopping and best-weight restoration to the tsai and proposed-LSTM experiments.
7. Correct the mixed-precision initialization and compare float32 with mixed float16 under otherwise identical settings.
8. Repeat each final configuration with at least five training seeds and report mean $\pm$ standard deviation.
9. Evaluate multiple held-out setup combinations or grouped cross-validation to test sensitivity to setup IDs 5 and 8.
10. Save per-sample predictions so that paired tests such as McNemar’s test can assess whether the 13-sample teacher-to-LSTM improvement and one-sample tsai-to-LSTM difference are statistically reliable.
11. Evaluate probability averaging or stacking between tsai and the LSTM because their errors on classes 12 and 16 are complementary.

10 Conclusion

The proposed 1DCNN–LSTM–ResNet pipeline provides a more physically natural time-series representation and improves generalization in the observed run. Compared with the teacher-reference adaptation, it increases test accuracy from 56.18% to 60.00%, increases macro F1 from 53.63% to 55.02%, and reduces the train–test accuracy gap. These improvements are consistent with the addition of regularization and the reduction in model capacity, but the experiment does not isolate their individual effects.

The cost is substantial: the proposed model is approximately 10.7 times slower per steady-state epoch because it processes a 3,000-step recurrent sequence instead of an approximately 14-step sequence. Its gain also depends heavily on classes 4 and 16, while performance deteriorates for classes 0, 3, 5, and 12 and remains zero for class 15. Therefore, the most defensible conclusion is that the proposed pipeline offers a modest but uneven generalization improvement at a large computational cost. Controlled ablations, repeated runs, and a corrected mixed-precision experiment are required before attributing the improvement to a specific design decision.

The tsai ResNet provides the strongest macro-F1 result, 57.67%, and essentially ties the proposed LSTM in test accuracy while using one GPU. It is therefore a competitive and computationally simpler baseline for this dataset. However, it reaches 100% train accuracy, has a larger train–test gap than the proposed LSTM, and completely misses classes 15 and 16. The current evidence supports retaining both models for further study rather than declaring one universally superior: tsai is preferable by macro F1 and single-GPU efficiency, while the proposed LSTM exhibits less overfitting and uniquely recognizes class 16.